

STEM BUILDERS



Python Overview

1. PYTHON OVERVIEW

What is Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands. Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, Unix shell, and other scripting languages. Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL). Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python Features

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.

- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries, and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java. Python

Setting up Environment

Installing Python

STEM Builders prefer Spyder as IDE for Python programming. Faculty can use any other preferred editor as well. All programs can be open and execute in any editor.

Spyder is a free and open source scientific environment written in Python, for Python, and designed by and for scientists, engineers and data analysts. It features a unique combination of the advanced editing, analysis, debugging, and profiling functionality of a comprehensive development tool with the data exploration, interactive execution, deep inspection, and beautiful visualization capabilities of a scientific package.

Download and install appropriate Spyder package from:
<https://docs.spyder-ide.org/current/installation.html>

For more details about Spyder please visit:
<https://www.spyder-ide.org>

Environment Setup

There are two ways to set up your python environment:

Use directly the standalone installers for Windows:

(https://github.com/spyder-ide/spyder/releases/latest/download/Spyder_64bit_full.exe)

and macOS: (<https://github.com/spyder-ide/spyder/releases/latest/download/Spyder.dmg>)

Use the Anaconda Python Distribution: (<https://www.anaconda.com/products/individual>) which includes Spyder by default. Once you download Anaconda, open the Anaconda Navigator (Downloads folder). In the pop up window, scroll to Spyder under Home and click Launch.

Spyder Overview

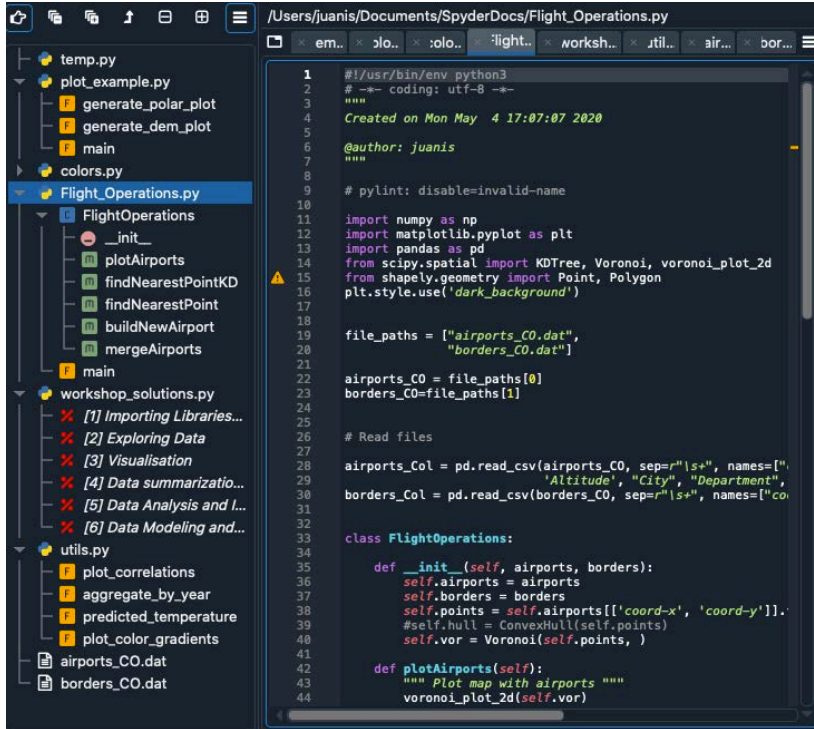
Spyder's multi-language Editor integrates a number of powerful tools right out of the box for an easy to use, efficient editing experience. The Editor's key features include syntax highlighting (pygments); real-time code and style analysis (pyflakes and pycodestyle); on-demand completion, calltips and go-to-definition features (rope and jedi); a function/class browser, horizontal and vertical splitting, and much more.

```

1  """
2  Plot a terrain model and a polar plot side by side.
3  """
4
5  # Third party imports
6  import numpy as np
7  import matplotlib.pyplot as plt
8  import matplotlib.cm
9  import matplotlib.colors
10 import mpl_toolkits.mplot3d # pylint: disable=unused-import
11
12 plt.style.use("dark_background")
13
14 def generate_polar_plot():
15     """Generate an example polar slice plot."""
16     # Compute pie slices
17     n_slices = 20
18     theta = np.linspace(0.0, 2 * np.pi, n_slices, endpoint=False)
19     radii = 10 * np.random.rand(n_slices)
20     width = np.pi / 4 * np.random.rand(n_slices)
21
22     fig, ax = plt.subplots(figsize=(8, 3))
23     fig.patch.set_facecolor('#395979')
24     ax1 = plt.subplot(1, 2, 2, projection='polar')
25     ax1.set_facecolor('#395979')
26     bars = ax1.bar(theta, radii, width=width, bottom=0.0)
27
28     # Use custom colors and opacity
29     for radius, plot_bar in zip(radii, bars):
30         plot_bar.set_facecolor(plt.cm.viridis(radius / 10.))
31         plot_bar.set_alpha(0.5)
32
33 def generate_dem_plot():
34     """Generate a 3D representation of a terrain DEM."""
35
36
91
92
93 _colors_full_map = {}
94 # Set by reverse priority order.
95 _colors_full_map.update(XKCD_COLORS)
96 _colors_full_map.update({k.replace('grey', 'gray'): v
97                          for k, v in XKCD_COLORS.items()
98                          if 'grey' in k})
99 _colors_full_map.update(CSS4_COLORS)
100 _colors_full_map.update(TABLEAU_COLORS)
101 _colors_full_map.update({k.replace('grey', 'gray'): v
102                          for k, v in TABLEAU_COLORS.items()
103                          if 'gray' in k})
104 _colors_full_map.update(BASE_COLORS)
105 _colors_full_map = _ColorMapping(_colors_full_map)
106
107
108 def get_named_colors_mapping():
109     """Return the global mapping of names to named colors."""
110     return _colors_full_map
111
112
113 def _sanitize_extrema(ex):
114     if ex is None:
115         return ex
116     try:
117         ret = ex.item()
118     except AttributeError:
119         ret = float(ex)
120     return ret
121
122
123 def _is_nth_color(c):
124     """Return whether *c* can be interpreted as an item in the color cycle."""
125     return isinstance(c, str) and re.match(r"AC[0-9]+Z", c)
126
127
128 def _is_color_like(c):
129     """Return whether *c* can be interpreted as an RGB(A) color."""
130     # Special-case nth color syntax because it cannot be parsed during setup.
131     if _is_nth_color(c):
132         return True
133     try:
134         to_rgba(c)
135     except ValueError:
136         return False
137

```

Spyder's Editor panel, split horizontally and with style analysis
Outline Explorer (function/class/method browser) and horizontal/vertical splitting capabilities:



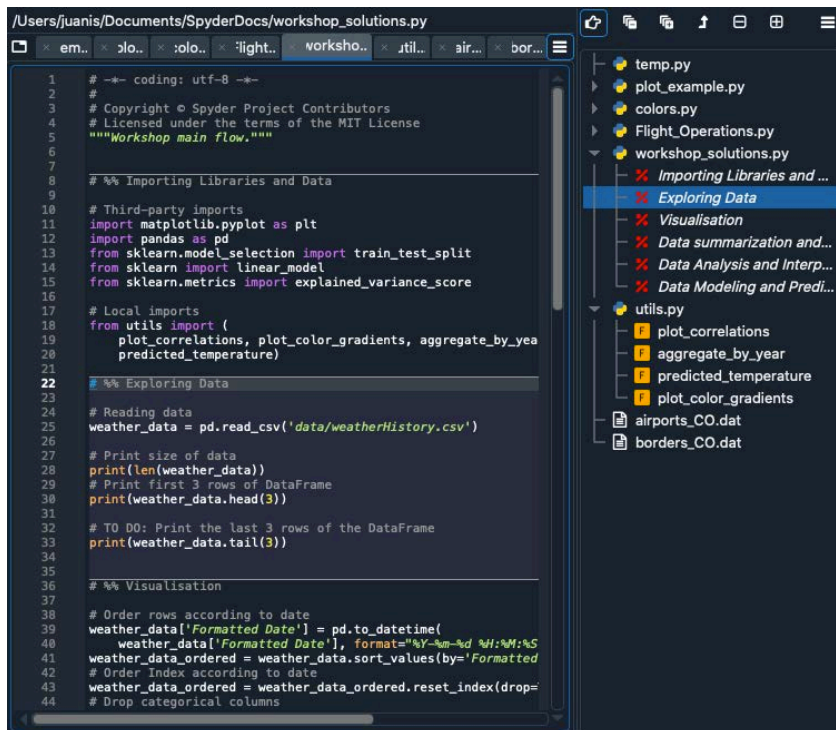
```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Created on Mon May  4 17:07:07 2020
5
6  @author: juanis
7  """
8
9  # pylint: disable=invalid-name
10
11 import numpy as np
12 import matplotlib.pyplot as plt
13 import pandas as pd
14 from scipy.spatial import KDTree, Voronoi, voronoi_plot_2d
15 from shapely.geometry import Point, Polygon
16 plt.style.use('dark_background')
17
18
19 file_paths = ["airports_CO.dat",
20              "borders_CO.dat"]
21
22 airports_CO = file_paths[0]
23 borders_CO=file_paths[1]
24
25
26 # Read files
27
28 airports_CO = pd.read_csv(airports_CO, sep=";", names=["
29 'Altitude', 'City', 'Department',
30 borders_CO = pd.read_csv(borders_CO, sep=";", names=["co
31
32
33 class FlightOperations:
34
35     def __init__(self, airports, borders):
36         self.airports = airports
37         self.borders = borders
38         self.points = self.airports[['coord-x', 'coord-y']]
39         #self.hull = ConvexHull(self.points)
40         self.vor = Voronoi(self.points, )
41
42     def plotAirports(self):
43         """ Plot map with airports """
44         voronoi_plot_2d(self.vor)
  
```

A snippet of code in the Spyder Editor, showing code style warnings

Defining code cells

A “code cell” in Spyder is a block of lines, typically in a script, that can be easily executed all at once in the current IPython Console. This is much like a “cell” in MATLAB (except without any need to enable a “cell mode”, since in Spyder, cells are detected automatically). You can divide your scripts into as many cells as needed, or none at all—the choice is yours.



```

1  # -*- coding: utf-8 -*-
2  #
3  # Copyright © Spyder Project Contributors
4  # Licensed under the terms of the MIT License
5  """Workshop main flow."""
6
7
8  %% Importing Libraries and Data
9
10 # Third-party imports
11 import matplotlib.pyplot as plt
12 import pandas as pd
13 from sklearn.model_selection import train_test_split
14 from sklearn import linear_model
15 from sklearn.metrics import explained_variance_score
16
17 # Local imports
18 from utils import (
19     plot_correlations, plot_color_gradients, aggregate_by_year,
20     predicted_temperature)
21
22 %% Exploring Data
23
24 # Reading data
25 weather_data = pd.read_csv('data/weatherHistory.csv')
26
27 # Print size of data
28 print(len(weather_data))
29 # Print first 3 rows of DataFrame
30 print(weather_data.head(3))
31
32 # TO DO: Print the last 3 rows of the DataFrame
33 print(weather_data.tail(3))
34
35
36 %% Visualisation
37
38 # Order rows according to date
39 weather_data['Formatted Date'] = pd.to_datetime(
40     weather_data['Formatted Date'], format="%Y-%m-%d %H:%M:%S")
41 weather_data_ordered = weather_data.sort_values(by='Formatted
42 # Order Index according to date
43 weather_data_ordered = weather_data_ordered.reset_index(drop=
44 # Drop categorical columns
  
```

You can separate cells by lines starting with either:

%% (standard cell separator)

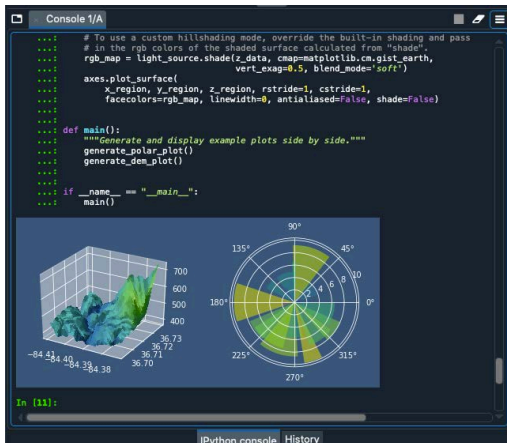
%% (standard cell separator, when file has been edited with Eclipse)

<codecell> (IPython notebook cell separator)

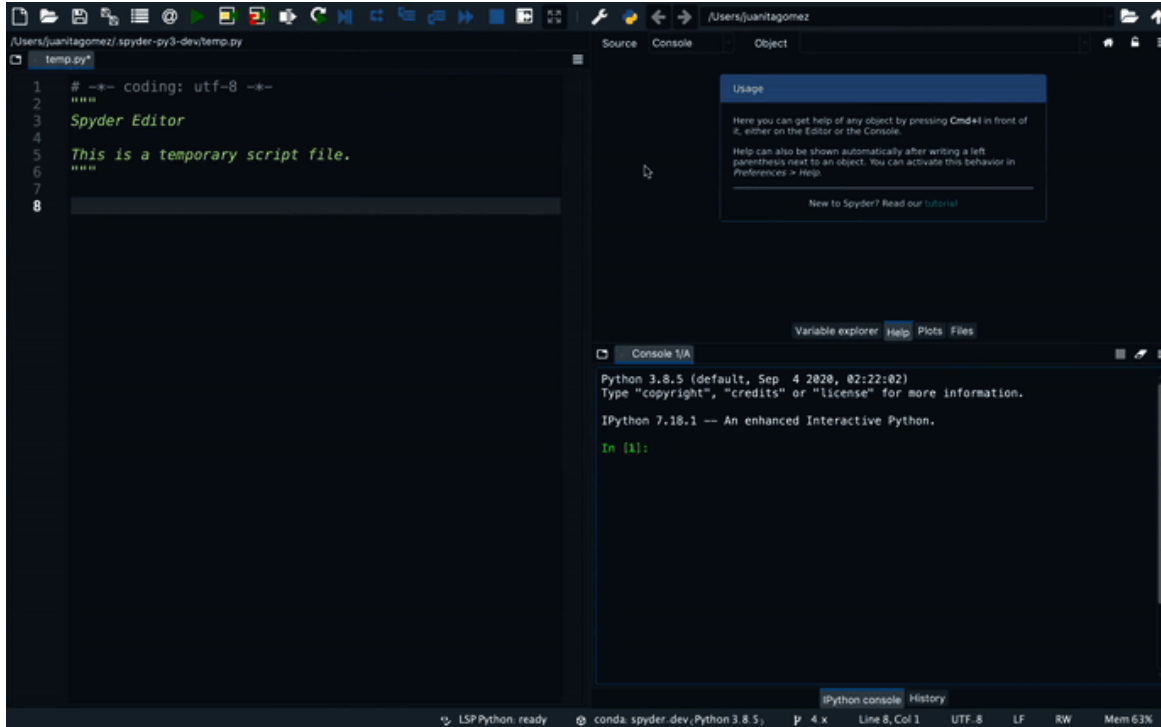
Providing a description to the right of the separator will give that cell its own name in the Outline Explorer.

IPython Console

The IPython Console allows you to execute commands and interact with data inside IPython interpreters.



To launch a new IPython instance, go to New console (default settings) under the Consoles menu, or use the keyboard shortcut Ctrl-T (Cmd-T on macOS) when the console is focused.



From the same menu, you can stop currently executing code with Interrupt kernel, clear a console's namespace with Remove all variables, or relaunch a fresh one with Restart kernel. As each console is executed in a separate process, this won't affect any others you've opened, and you will be able to easily test your code in a clean environment without disrupting your primary session.

Supported features

Any IPython Console, whether external or started by Spyder, supports:

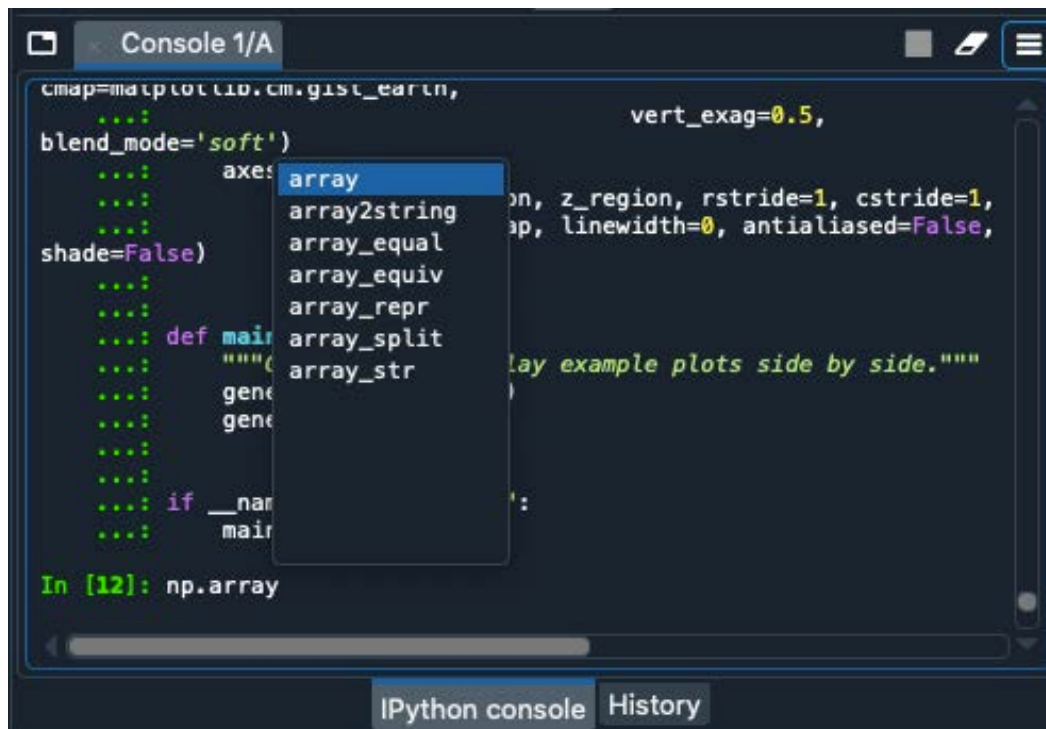
Automatic code completion

Real-time function calltips

Full GUI integration with the enhanced Spyder Debugger.

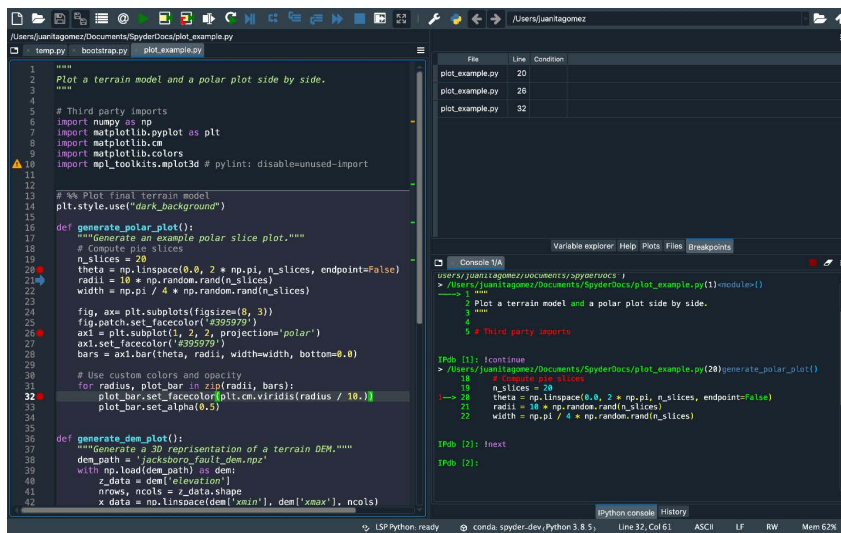
The Variable Explorer, with GUI-based editors for many built-in and third-party Python objects.

Display of Matplotlib graphics in Spyder's Plots pane, if the Inline backend is selected under Preferences IPython console Graphics Graphics backend, and inline in the console if Mute inline plotting is unchecked under the Plots pane's options menu.



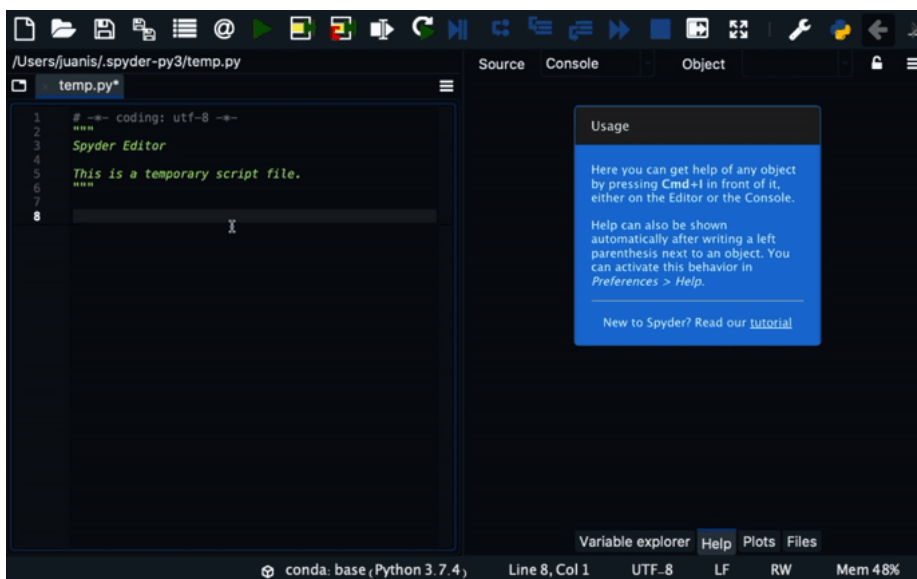
Debugger

Debugging in Spyder is supported through integration with the enhanced ipdb debugger in the IPython Console. This allows breakpoints and the execution flow to be viewed and controlled right from the Spyder GUI, as well as with all the familiar IPython console commands.



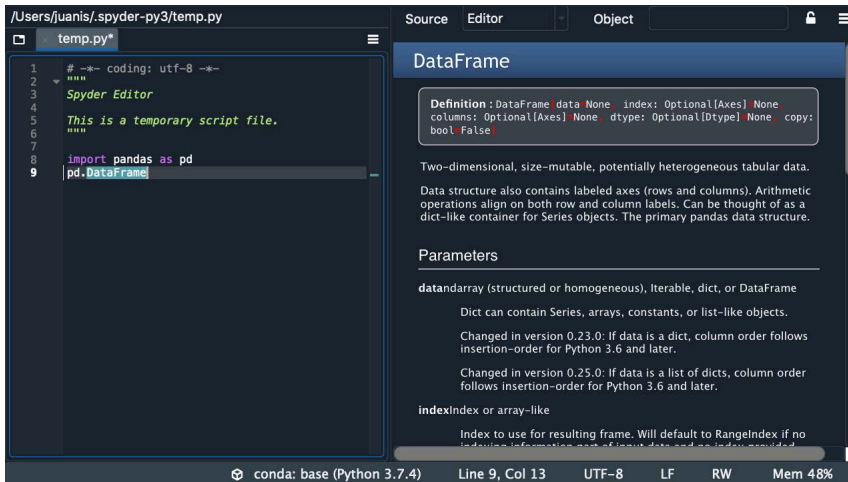
Help

You can use the Help pane to find, render and display rich documentation for any object with a docstring, including modules, classes, functions and methods. This allows you to access documentation easily directly from Spyder, without having to interrupt your workflow.



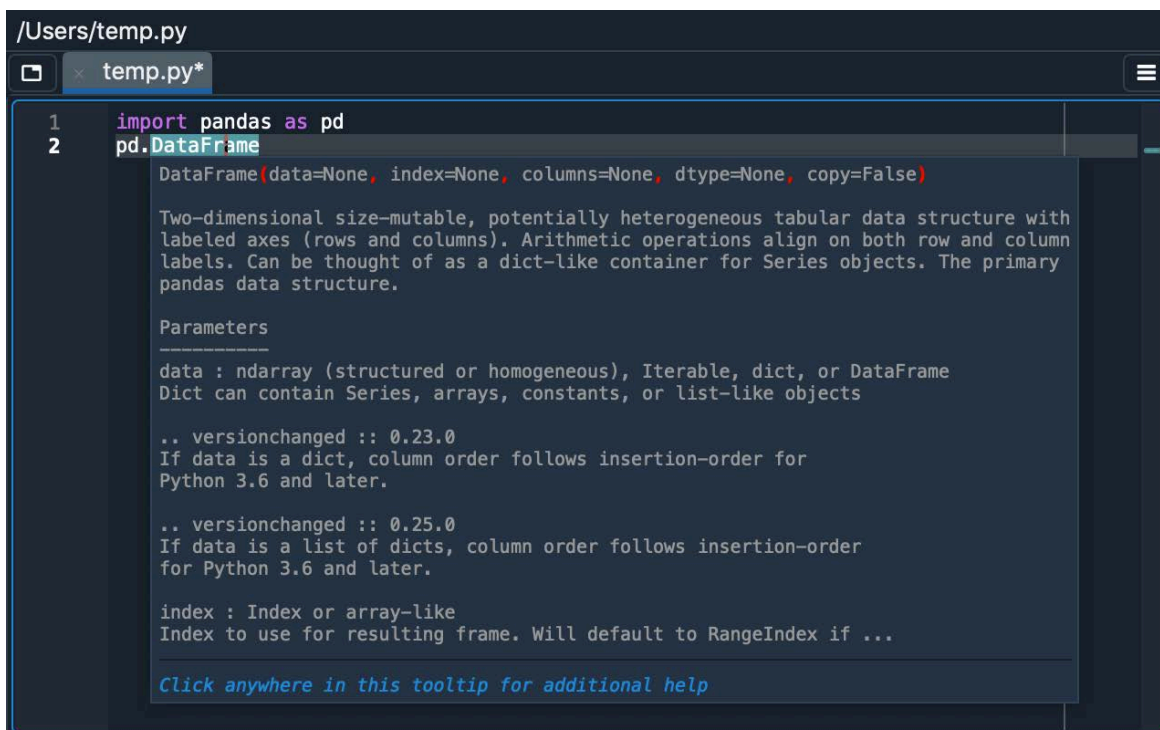
Getting help

Help can be retrieved both by static analysis of open files in the Editor, or by dynamically inspecting an object in an IPython Console. You can trigger help manually by pressing the configurable help shortcut (Ctrl-I by default).



Getting help by hovering

You can also get summary help for objects by hovering over them in the Editor. Clicking the hover popup will open the full documentation in the Help pane.



Writing my First Program

"Welcome to STEM Builders!" in Python

When programmers are learning a new language, we tend to write a one-line program that prints some version of the message "Hello world!" this is a simple program that shows whether your computer is properly set up to run Python programs.

```
print("Welcome to STEM Builders!")
```

output

Welcome to STEM Builders!

Note: If you are using Python 2.7, this would be:

```
print "Welcome to STEM Builders!"
```

output

Welcome to STEM Builders!

If it works, congratulations! You just ran your first Python program.



Motivate to Innovate...



**STEM
BUILDERS**
Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS



contactus@stembuilders.com



www.stembuilders.com